Haskell Notes

目次

1 凡例 2
2 Haskell について 3
2.1 Haskell という森 3
2.2 関数型プログラミング 4
2.3 関数型という考え方 5
3 変数・関数・型 6
3.1 変数 6
3.2 変数の型 7
3.3 関数 7
3.4 有名な関数 7
3.5 関数の型 8
3.6 関数合成 8
3.7 IO サバイバルキット 1 8
4 条件分岐と再帰呼び出し 9
4.1 ラムダ 9
4.2 ローカル変数 9
4.3 クロージャ 10
4.4 型 10
4.5 条件 11
5 Test Part 12
5.1 Function application 12
5.2 List map 12
5.3 Maybe map 12
5.4 Functor map 12
5.5 Applicative map 12
5.6 Apllicative map 2 12
5.7 Monadic function application 13
5.8 Bind 13
5.9 Double bind 13
5.10 Contextual map 13
6 Test Part 2 13
6.1 Lambda 13
6.2 Operators 13
6.3 Types 14

1 凡例

種類	字体・表記法	例
変数・関数	イタリック（1 文字）	x, f
有名な変数・関数・定数	ローマン・大文字	$\mathrm{first}, \mathrm{id}, \mathrm{otherwise}$
リスト変数	変数名に s をつける	x_{s}
Maybe 変数	変数名に ? をつける	$x_?$
一般のコンテナ変数	変数名に $*$ をつける	x_*
定数値コンストラクタ	ローマン・大文字	$\mathrm{True}, \mathrm{Nothing}$
値コンストラクタ	ローマン・大文字	Just_x
有名な定数値コンストラクタ	数学記号	$\emptyset, \oslash$
有名な値コンストラクタ	特別なカッコで包む	$[x], (y)$
アクション	ギリシア文字（1 文字）	α, μ
有名なアクション	サンセリフ	$\mathsf{main}, \mathsf{print}$
型（引数なし）	ボールドイタリック（1 文字）	$\boldsymbol{a}$
型（引数あり）	ボールドイタリック（1 文字）	$\boldsymbol{m_a}$
有名な型（引数なし）	ボールド・大文字	$\mathbf{Int}$
有名な型（引数あり）	特別なカッコで包む	$[\boldsymbol{a}], [\mathbf{Int}]$
型クラス	フラクチュール	$\mathfrak{Num}$
キーワード	固定幅	`let`
無名パラメタ	ひし形（白）	$\lozenge$
無名型パラメタ	ひし形（黒）	$\blacklozenge$
集合（数学）	ブラックボード（1 文字）	$\mathbb{Z}$
関手（数学）	スクリプト（1 文字）	$\mathscr{F}$
圏（数学）	カリグラフィ（1 文字）	$\mathcal{C}$

2 Haskell について

本書はプログラミング言語 Haskell の入門書である．それと同時に，本書はプログラミング言語を用いた代数構造の入門書でもある．プログラミングと代数構造の間には密接な関係があるが，とくに**関数型プログラミング**を実践する時にはその関係を意識する必要が出てくる．本書はその両者を同時に解説することを試みる．

2.1 Haskell という森

これからのプログラマにとって Haskell を無視することはできない．Haskell の「欠点をあげつらうことも，攻撃することもできるが，無視することだけはできない」のだ．それは Haskell がプログラミングの本質に深く関わっているからである．

Haskell というプログラミング言語を知ろうとすると，従来のプログラミング言語の知識が邪魔をする．モダンで，人気があって，Haskell から影響を受けた言語，たとえば Ruby や Swift の知識さえ，Haskell を学ぶ障害になり得る．ではどのようにして Haskell の深みに到達すればいいのだろうか．

その答えは，一見遠回りに見えるが，一度抽象数学の高みに登ることである．

と言っても，あわてる必要はない．

近代的なプログラミング言語を知っていれば，すでにある程度抽象数学に足を踏み入れているからである．そこで，本書では近代的なプログラマを対象に，プログラミング言語を登山口に抽象数学の山を登り，その高みから Haskell という森を見下ろすことにする．

さて，登山口にどのプログラミング言語を選ぶのが適当であろうか．TIOBE Index 2023 年 2 月版によると「ビッグ 5」として Python，C，C++，Java，C# が挙げられている．[1]順位の変動はあるが，他の調査でもビッグ 5 は過去何年も変動していないので，当座は妥当な統計であろう．このうち C は「多くのプログラマが読める」以外にメリットが無く，その唯一のメリットさえ最近は怪しくなっているため，登山口候補から外す．残るは Java，C# グループと Python ということになるが，シンプルであり，かつ Haskell と対極にある言語である Python を登山口に選ぶことにした．

本書では Python コードはこのように登場する．

```
print("Hello, world.")
```

本書に示すコードは擬似コードではなく，すべて実行可能な本物のコードである．

一部の章でどうしても型に触れないといけない部分がある．Python は動的型付け言語であり，型の説明には不適切であるため，この部分だけ理解の助けとして C++によるコードを例示した．この部分はコードを読まなくても先に進める．

ところで，プログラムのソースコードは現代でも ASCII 文字セットの範囲で書くことが標準的である．Unicode を利用したり，まして文字にカラーを指定したり，書体や装飾を指定することは一般的ではない．たとえば変数 a のことを a と書いたり $\boldsymbol{a}$ と書いたり $\tilde{a}$ と書いたりして区別することはない．

Haskell プログラマもまた，多くの異なる概念を同じ貧弱な文字セットで表現しなければならない．これは，はじめて Haskell コードを読むときに大きな問題になりえる．たとえば Haskell では `[a]` という表記をよく扱う．この `[a]` は `a` という変数 1 要素からなるリストのこともあるし，`a` 型という仮の型から作ったリスト型の場合もあるが，字面からでは判断できない．も

[1] https://www.tiobe.com/tiobe-index/

し変数はイタリック体，型はボールド体と決まっていれば，それぞれ $[a]$ および $[\boldsymbol{a}]$ と区別できたところである．

本書は，異なる性質のものには異なる書体を割り当てるようにしている．ただし，どの表現もいつでも Haskell に翻訳できるように配慮している．実際，本書執筆の最大の困難点は，数学的に妥当で，かつ Haskell の記法とも矛盾しない記法を見つけることであった．

2.2 関数型プログラミング

プログラマはなぜHaskellを習得しなければならないのだろう．それはHaskellと**関数型プログラミング**の間に密接な関係があるからである．

関数型プログラミングとはプログラミングにおける一種のスローガンのようなもので，どの言語を用いたから関数型でどの言語を用いたから関数型ではない，というものではない．しかし，関数型プログラミングを強くサポートする言語と，そうでない言語とがある．ここら辺の事情はオブジェクト指向プログラミングとプログラミング言語の関係と似ている．Haskell は関数型プログラミングを強くサポートし，Python はほとんどサポートしない．

関数型プログラミングの特徴を一言で言えば，プログラム中の**破壊的代入**を禁止することである．変数 x に 1 という数値が一度代入されたら，変数 x の値をプログラム中に書き換える，すなわち破壊的代入をすることはできない．この結果，変数の値はプログラムのどこでも，どの時点で読み出しても同じであることが保証される．これを変数の**参照透過性**と呼ぶ．

プログラム全体に参照透過性があると，そのプログラムはブロックに分割しやすく，各々のブロックは再利用しやすい．またプログラムのどの断片から読み始めても，全体の構造を見失いにくい．これが関数型プログラミングとそれを強くサポートする Haskell を習得する理由である．

参照透過性がもたらすもう一つのボーナスは変数の**遅延評価**である．変数はいつ評価しても値が変わらないのだから，コンパイラは変数をできるだけ遅く評価してよい．この遅延評価によって，Haskell コンパイラは他の言語に見られない**無限リスト**を扱う能力を獲得している．

ここで数学とプログラミングの関係について述べておこう．ある方程式を解くためにコンピュータによって数値シミュレーションを行うとか，非常に複雑な微分を機械的に行うとか，プログラミングによって数学をサポートすることは計算機科学の主たる分野の一つであるが，ここではもっと根源的な話をする．

数学者もプログラマも**関数**をよく使う．数学者が使う関数とは，引数がいくつかあって，その結果決まる戻り値があるようなものだ．一方でプログラマが使う関数というのは，引数と戻り値はだいたい同じとして，中身に条件分岐があったり，ループがあったり，外部変数を書き換えたり，入出力をしたりする．

どちらも同じ関数であるのに，なぜこうもイメージが違うのだろうか．

もし，我々が関数型プログラミングの原則を忠実に守り，プログラム中のいかなる破壊的代入をも禁止するとすると，両者の関数は全く同じ性格になる．逐次実行も条件分岐もループも，それどころか定数さえ，**ラムダ式**という式だけで書けるようになる．あらゆるプログラムが，最終的には単一のラムダ式で書ける．

ところが，入出力，状態変数，例外など，プログラミングに使われる多くのテクニックは関数の副作用を前提としている．参照透過性と副作用を統一的に扱うためには**モナド**という数学概念が必要である．Haskell はモナドを陽に扱うプログラミング言語である．

2.3 関数型という考え方

関数型プログラミングという考え方自体は，プログラミングに必ずしも束縛されるものではない．

たとえばいま，ある英文テキストファイルから，単語を出現頻度順に取り出したいとしよう．こんなとき，プログラマならば次のように考えるのではないだろうか．

「まずは英文テキスト中の大文字を全部小文字にしよう」「それからアルファベットと空白以外を全部捨ててしまおう」「空白を改行に置き換えると，1 行に 1 単語になるな」「ここまで来たら出現頻度順に並べ直すのはわけないことだ」……

これらの言葉を UNIX の言葉に置き換えてみる．ここでは英文テキストを `the-great-gatsby.txt` としておこう．中身はフィッツジェラルドの名作「The Great Gatsby」（華麗なるギャッツビー）だ．最初の「まずは英文テキスト中の大文字を全部小文字にしよう」は UNIX コマンドではこうなる．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]'
```

この出力を適当なファイルにリダイレクトすれば，大文字がすべて小文字に置き換わったテキストファイルになる．

次にするのは「それからアルファベットと空白以外を全部捨ててしまおう」だ．UNIX コマンドではこうなる．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]'
```

これで単語だけのテキストファイルが出来る．次に空白をすべて改行に置き換えてしまおう．UNIX コマンドでこうする．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
```

1 行に 1 単語ずつ並べられたテキストファイルがこれで手に入ることになる．空行もあるが，それは問題ない．このファイルの中身をアルファベット順にソートする．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort
```

ソートできれば，同一の連続する行を数え上げて，一つの行にすればいい．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c
```

最後に，出現頻度順に逆順ソートをかけると「出現頻度順」になる．それには以下のようにする．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c | sort -nr
```

この出力の先頭 10 行は，そのまま出現頻度上位 10 傑の単語となる．

さて，この出現頻度上位単語の抽出のどこが「関数型プログラミング」の考え方だったのだろうか．

実はこんな風に考えることが出来る．入力を x とする．いまの例では x がファイル `the-great-gatsby.txt` である．出力を y とする．ここに y は出現頻度順に並んだ出現頻度と単語のリストである．僕たちが欲しいのは

$$y = f(x) \tag{1}$$

となるようなプログラム f だった．プログラム f は簡単には手に入らないので，僕たちは UNIX コマンドの `tr` とか `sort` とか `uniq` とかをつなぎ合わせて作った．それらを「数学風」に書くと次のようになる．

$$\begin{aligned} f_1 &= \mathrm{tr}_{[A\ldots Z]\to[a\ldots z]} \\ f_2 &= \mathrm{tr}_{\overline{[a\ldots z,\square]}\to\emptyset} \\ f_3 &= \mathrm{tr}_{\square\to\swarrow} \\ f_4 &= \mathrm{sort} \\ f_5 &= \mathrm{uniq}_c \\ f_6 &= \mathrm{sort}_{n,r} \end{aligned} \tag{2}$$

そして x から y を得るために $y = f_6(f_5(f_4(f_3(f_2(f_1(x))))))$ という計算を行った．カッコが多すぎるので，この式を$y = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1(x)$ と書き直そう．ここに演算子 $\circ$ は「**関数合成演算子**」だ．

関数合成演算子を使うと，プログラム f を $f = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1$ と定義することも出来る．これが何を意味しているかと言うと，プログラムを小さな部分プログラムに分解したということだ．

分解したなら，合成する方法が必要になる．この例では，プログラムの各部分が他の部分に依存していないために，合成は数学的な関数合成と同じ方法が使える．

上記の例のように，数学における関数合成をそのままプログラミングに持ち込めれば，部分プログラムの合成も見通しが良いものになる．このような考え方が「関数型プログラミング」のエッセンスなのである．

TK

3 変数・関数・型

3.1 変数

変数 x に値 1 を代入するには次のようにする．[2]

$$x = 1 \tag{3}$$

変数という呼び名に反して，変数の値は一度代入したら変えられない．そこで変数に値を代入するとは呼ばずに，変数に値を**束縛**するという． 式 3 の右辺のように数式にハードコードされた値を**リテラル**と呼ぶ．

[2] Haskell では `x = 1` と書く．

リテラルや変数には**型**がある．型は数学者の**集合**と似た意味で，整数全体の集合 $\mathbb{Z}$ に相当する**整数型**や，実数全体の集合 $\mathbb{R}$ に相当する**浮動小数点型**がある．整数と整数型，実数と浮動小数点型は異なるため，整数型を **Int** で，浮動小数点型を **Double** で表すことにする．[3]

数学者は変数 x が整数であることを $x \in \mathbb{Z}$ と書くが，本書では $x :: \mathbf{Int}$ と書く．これは記号 $\in$ を別の用途に用いるためである．[4]

本書では変数名を原則 1 文字として，イタリック体で表し w, x, y, z のような n 以降のアルファベットを使う．

変数の値がいつでも変化しないことを**参照透過性**と呼ぶ．プログラマが変数の値を変化させたい，つまり**破壊的代入**を行いたい理由はユーザ入力，ループ，例外，内部状態の変化，大域ジャンプ，継続を扱いたいからであろう．しかし，後に見るようにループ，例外，内部状態の変化，大域ジャンプ，継続に変数の破壊的代入は必要ない．ユーザ入力に関しても章を改めて取り上げる．参照透過性を強くサポートするプログラミング言語をを**関数型プログラミング言語**と呼ぶ．

3.2 変数の型

TK

$$x :: \mathbf{Int} \tag{4}$$

$$x = 1 \tag{5}$$

$$x :: \mathbf{Int} = 1 \tag{6}$$

3.3 関数

整数 x に 1 を足す**関数** f は次のように定義できる．

$$fx = x + 1 \tag{7}$$

ここに x は関数 f の**引数**である．引数は括弧でくるまない．[5]

本書では関数名を原則 1 文字として，イタリック体で表し，f, g, h のようにアルファベットの f 以降の文字を使う．ただし有名な関数についてはローマン体で表し，文字数も 2 文字以上とする．たとえば sin などの三角関数や指数関数がそれにあたる．

変数 x に関数 f を**適用**する場合は次のように書く．ここでも引数を括弧でくるまない．[6]

$$z = fx \tag{8}$$

関数 f が引数をふたつ取る場合は，次のように書く．[7]

$$z = fxy \tag{9}$$

なお fxy は $(fx)y$ と解釈される．前半の (fx) は 1 引数の関数とみなせる．2 引数関数を連続した 1 引数関数の適用とみなす考え方を，関数の**カリー化**と呼ぶ．

3.4 有名な関数

TK

[3] Haskell では `Int` および `Double` と書く．
[4] Haskell では $x :: \mathbf{Int}$ を `x :: Int` と書く．
[5] Haskell では `f x = x + 1` と書く．
[6] Haskell では `z = f x` と書く．
[7] Haskell では `z = f x y` と書く．

3.5 関数の型

TK

3.6 関数合成

変数 x に関数 f と関数 g を連続して適用したい場合 $z = g(fx)$ とするところであるが，事前に関数 f と関数 g を**合成**しておきたいことがある．

関数の合成は次のように書く．

$$k = g \circ f \tag{10}$$

関数の連続適用 $g(fx)$ と合成関数の適用 $(g \circ f)x$ は同じ結果を返す．[8]

関数合成演算子 . は以下のように**左結合**する．

$$\begin{aligned} k &= h \circ g \circ f \\ &= (h \circ g) \circ f \end{aligned} \tag{11}$$

関数適用のための特別な演算子 $ があると便利である．演算子 $ は関数合成演算子よりも優先順位が低い．例を挙げる．[9]

$$\begin{aligned} z &= h \circ (g \circ f)x \\ &= h \,\$\, g \circ fx \end{aligned} \tag{12}$$

3.7 IO サバイバルキット 1

プログラムとは合成された関数である．多くのプログラミング言語では，プログラムそのものに main という名前をつける．本書では「**IO モナド**」の章で述べる理由によって，main 関数をサンセリフ体で main と書く．

実用的なプログラムはユーザからの入力を受け取り，関数を適用し，ユーザへ出力する．Haskell ではユーザからの 1 行の入力を getLine で受け取り，変数の値を print で書き出せる．ここに getLine と print は関数（ファンクション）ではあるが，特別に「**アクション**」とも呼ぶ．関数 main もアクションである．

引数 x の 2 乗を求める関数 f は次のように定義できる．

$$\begin{aligned} f &:: \mathbf{Double} \to \mathbf{Double} \\ fx &= x \times x \end{aligned} \tag{13}$$

ユーザからの入力に関数 f を適用してユーザへ出力するプログラムを Haskell で書くと次のようになる．

$$\mathsf{main} = \mathsf{print} \circ f \circ \mathrm{read} \heartsuit \mathsf{getLine} \tag{14}$$

ここに関数 read は**文字列**であるユーザ入力を数に変換する関数である．また演算子 ♡ は新たな関数合成演算子で，アクションとアクションを合成するための特別な演算子である．詳細は「**モナド**」の章で述べる．

- Haskell では 式 13 と式 14 をまとめて次のように書く．

```
f :: Double -> Double
```

[8] Haskell では $k = g \circ f$ を `k = g . f` と書く．
[9] Haskell では $z = h \,\$\, g \circ fx$ を `z = h $ (g . f) x` と書く．

```
f x = x * x

main = print . f . read =<< getLine
```

4 条件分岐と再帰呼び出し

4.1 ラムダ

関数とは，変数名に束縛された**ラムダ式**である．引数をひとつとり，その引数に 1 を足して返す関数 f はラムダ式を用いて次のように書ける．

$$f = \backslash\, x \mapsto x + 1 \tag{15}$$

ラムダ式は入れ子に出来る．引数をふたつとり，その引数同士を足すラムダ式は次のように書ける．

$$\backslash\, x \mapsto (\backslash\, y \mapsto x + y) \tag{16}$$

より簡潔に式 16 を

$$\backslash\, xy \mapsto x + y \tag{17}$$

と書いても良い．

本書では無名変数 $\diamond$ を用いた以下の書き方も用いる．[10]

$$\begin{aligned} f &= (\diamond + 1) \\ &= \backslash\, x \mapsto x + 1 \end{aligned} \tag{18}$$

無名変数が 2 回以上登場した場合は，その都度新しいパラメタを生成する．たとえば次のとおりである．[11]

$$\begin{aligned} f &= \diamond + \diamond \\ &= \backslash\, x \mapsto \backslash\, y \mapsto x + y \\ &= \backslash\, xy \mapsto x + y \end{aligned} \tag{19}$$

4.2 ローカル変数

関数内で**ローカル変数**を使いたい場合は以下のように行う．[12]

$$z = \texttt{let}\ y \equiv 1\ \texttt{in}\ x + y \tag{20}$$

ローカル変数はラムダ式のシンタックスシュガーである．式 20 は次の式と等価である．

$$z = (\backslash\, y \mapsto x + y)1 \tag{21}$$

ローカル変数の定義は次のように後置できる．[13]

$$z = x + y\ \texttt{where}\ y \equiv 1 \tag{22}$$

[10] 無名変数は Haskell には無いが，代わりに「セクション」という書き方ができる．式 $f = (\diamond + 1)$ を Haskell では `f = (+1)` と書く．

[11] Haskell では $f = (\diamond + \diamond)$ を `f = (+)` と書く．

[12] Haskell では $z = \texttt{let}\ y \equiv 1\ \texttt{in}\ x + y$ を `z = let y = 1 in x + y` と書く．

[13] Haskell では $z = x + y\ \texttt{where}\ y \equiv 1$ を `z = x + y where y = 1` と書く．

4.3 クロージャ

ラムダ式を返す関数は，ラムダ式内部に値を閉じ込めることができる．たとえば

$$fn = \backslash x \mapsto n + x \tag{23}$$

のように関数を定義して良い．関数 f に引数 n を与えると，新たな 1 引数関数が得られる．例を挙げる．

$$\begin{aligned} n &= 3 \\ g &= fn \end{aligned} \tag{24}$$

この例では，関数 g の中に値 $n = 3$ が閉じ込められているため $g1$ は 4 と評価される．値を閉じ込めたラムダ式を**クロージャ**と呼ぶ．

4.4 型

TK 一部前倒し

すべての変数，関数には**型**がある．代表的な型には整数型，浮動小数点型，ブール型，文字型がある．整数型を **Int** で，浮動小数点型を **Double** で表すことはすでに述べたとおりである．

Haskell には 2 種類の整数型がある．ひとつは**固定長整数型**で，もうひとつは**多倍長整数型**である．Haskell では前者を `Int` で，後者を `Integer` で表す．多倍長整数型はメモリの許す限り巨大な整数を扱えるので，整数全体の集合に近いのであるが，本書では主に固定長整数型を用いる．

浮動小数点型には**単精度浮動小数点型**と**倍精度浮動小数点型**があり，Haskell では前者を `Float` で，後者を `Double` で表現する．単精度浮動小数点型はめったに用いられないため，本書では浮動小数点型と言えば倍精度浮動小数点型を意味することにする．

論理型は論理値 True または False のいずれかしか値をとれない型である．論理型のことを **Bool**と書く．[14]

文字型を **Char** と書く．[15]

変数 x の型が **Int**のとき，以下のように**型注釈**を書く．[16]

$$x :: \mathbf{Int} \tag{25}$$

1 引数関数の型は次のように注釈できる．[17]

$$f :: \mathbf{Int} \to \mathbf{Int} \tag{26}$$

ここで関数 f は整数型の引数をひとつとり，整数型の値を返す．[18]

2 引数関数の方は次のように注釈できる．[19]

$$f :: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Int} \tag{27}$$

[14] Haskell ではブール型を `Bool` と書く．
[15] Haskell では Unicode 文字型を `Char` と書く．
[16] Haskell では `x :: Int` と書く．
[17] Haskell では `f :: Int -> Int` と書く．
[18] 正確には `->` は型コンストラクタである．無名の型引数を $\blacklozenge$ で表すと `->` は $(\blacklozenge \to \blacklozenge)$ という，ふたつの型引数を取る型コンストラクタである．
[19] Haskell では `f :: Int -> Int -> Int` と書く．

ここで関数 f は整数型の引数をふたつとり，整数型の値を返す．型 $\mathbf{Int} \to \mathbf{Int} \to \mathbf{Int}$ は $\mathbf{Int} \to (\mathbf{Int} \to \mathbf{Int})$ と解釈される．

$(\mathbf{Int} \to \mathbf{Int})$ 型の関数を受け取り $(\mathbf{Int} \to \mathbf{Int})$ 型の関数を返す関数は次の型を持つ．[20]

$$f :: (\mathbf{Int} \to \mathbf{Int}) \to (\mathbf{Int} \to \mathbf{Int}) \tag{28}$$

なお後半の括弧は省略可能なので次のように書いても良い．

$$f :: (\mathbf{Int} \to \mathbf{Int}) \to \mathbf{Int} \to \mathbf{Int} \tag{29}$$

Haskell ではすべての変数, 関数に型があり, 型はコンパイル時に決定されていなければならない．ただし，式から**型推論**が行える場合は型注釈を省略できる．

4.5 条件

条件分岐は次のように書く．[21]

$$z = \texttt{if}\ x > 0\ \texttt{then}\ x\ \texttt{else}\ (-x) \tag{30}$$

$$f = \texttt{case}\ x\ \texttt{of} \begin{cases} 1 \longrightarrow 1 \\ \sim\sim \longrightarrow 0 \end{cases} \tag{31}$$

この場合 $x \equiv 1$ ならば f は 1 を，そうでなければ f は 0 を返す．ここに $\sim\sim$ はすべてのパターンに一致する記号である. パターンマッチは上から順に行われる. 条件分岐の代わりに以下のような**パターンマッチ**も使える．[22]

関数定義にもパターンマッチを使える．[23]

$$\begin{aligned} f1 &= 1 \\ f\sim\sim &= 0 \end{aligned} \tag{32}$$

関数定義には次のように**ガード**と呼ばれる条件を付与することができる．[24]

$$\begin{aligned} fx \mid_{x>0} &= x \\ \mid_{\text{otherwise}} &= (-x) \end{aligned} \tag{33}$$

ここに otherwise は True の別名である．

ガードは上から順にマッチされる．

---

[20] Haskell では `f :: (Int -> Int) -> (Int -> Int)` と書く．

[21] Haskell では `z = if x > 0 then x else (-x)` と書く．

[22] Haskell では以下のように書くのが一般的である．

```
f = case x of 1 -> 1
              _ -> 0
```

[23] Haskell では次のように書く．

```
f 1 = 1
f _ = 0
```

[24] Haskell では次のように書く．

```
f x | x > 0     = x
    | otherwise = (-x)
```

5 Test Part

5.1 Function application

$$z = fx \tag{34}$$

```
z = f x
```

5.2 List map

$$z_{\mathrm{s}} = f * x_{\mathrm{s}} \tag{35}$$

```
zs = map f xs
```

or

```
zs = f `map` xs
```

5.3 Maybe map

$$z_{?} = f \circledast x_{?} \tag{36}$$

```
zm =  fmap f xm
```

or

```
zm =  f <$> xm
```

5.4 Functor map

$$z_{*} = f \circledast x_{*} \tag{37}$$

```
zm = f <$> xm
```

5.5 Applicative map

$$z_{*} = f_{*} \boxast x_{*} \tag{38}$$

```
zm = fm <*> xm
```

5.6 Apllicative map 2

$$z_{*} = g_{*} \boxast x_{*} \boxast y_{*} \tag{39}$$

```
zm = gm <*> xm <*> ym
```

Or,

$$z_{*} = g \circledast x_{*} \boxast y_{*} \tag{40}$$

```
zm = g <$> xm <*> ym
```

5.7 Monadic function application

$$z_* = fx \tag{41}$$

```
zm = f x
```

5.8 Bind

$$z_* = f \heartsuit x_* \tag{42}$$

```
zm = f =<< xm
```

5.9 Double bind

$$z_* = g \heartsuit f \heartsuit x_* \tag{43}$$

```
zm = g =<< f =<< xm
```

5.10 Contextual map

$$\mathsf{printEach} = \mathsf{print} \mathbin{⁂} x_{\mathrm{s}} \tag{44}$$

```
printEach = print `mapM` xs
```

6 Test Part 2

6.1 Lambda

$$(\backslash\, x \mapsto x + 1)y \tag{45}$$

```
(\x -> x + 1) y
```

6.2 Operators

Scalar:

$$z = fx \tag{46}$$

```
z = f x
```

List:

$$z_{\mathrm{s}} = f * x_{\mathrm{s}} \tag{47}$$

```
zs = f `map` xs
```

Maybe:

$$z_{\mathrm{m}} = f \circledast x_{\mathrm{m}} \tag{48}$$

```
zm = f <$> xm
```

Any functor:

$$z_* = f \circledast x_* \tag{49}$$

```
z = f <$> x
```

Applicative version:

$$z_* = (f) \boxast x_* \tag{50}$$

```
z = f <*> x
```

Composition.

$$h = g \circ f \tag{51}$$

```
h = f . g
```

Scalar to contextual:

$$z_* = \tilde{f} x \tag{52}$$

```
z = f x
```

Monadic composition.

$$\tilde{h} = \tilde{g} \heartsuit \tilde{f} \tag{53}$$

```
h = g =<< f
```

6.3 Types

Scaler type.

$$x :: \mathbf{Int} \tag{54}$$

Function type.

$$f :: \boldsymbol{a} \to \boldsymbol{b} \tag{55}$$

Functor type.

$$\begin{aligned} x_{\mathrm{s}} &:: [\mathbf{Int}] \\ x_* &:: \mathfrak{Monad} \supset \boldsymbol{m}, \mathfrak{Eq} \supset \boldsymbol{a} \Rightarrow \boldsymbol{m}_{\boldsymbol{a}} \\ f &:: (\boldsymbol{r} \to \blacklozenge)_{\boldsymbol{q}} \end{aligned} \tag{56}$$

型引数は添字として表現することにした.